

Qué pasó dentro de mi sistema operativo cuando programé un script con cron

Lo que observé en Ubuntu y la teoría de sistemas operativos que lo explica

Asignatura: Sistemas Operativos
Estudiante: Cindy Guzmán
Fecha: 29 de septiembre de 2026
Equipo: Ubuntu (Linux), «cindy-guzman-Inspiron-15-3520»

1. De qué trata este reporte

Escribí un script en Bash, `salud.sh`, que anota la memoria RAM usada, el espacio en disco y la carga del sistema, y lo programé con `cron` para que corra solo cada dos minutos. Aquí explico qué salió en la terminal y **por qué** el sistema operativo se comportó así, apoyándome en la teoría de la materia.

Ojo con los datos: solo tengo **dos registros** del script (17:00:01 y 17:02:01). Sirven para comparar contra la teoría, pero no para hablar de tendencias. Cuando algo es hipótesis y no algo que los datos demuestren, lo digo.

2. Qué hice

1. Creé `salud.sh` con `sudo gedit`, escribí el script y lo guardé.
2. Le di permisos con `sudo chmod 777 salud.sh` y revisé con `ls -l`.
3. Lo programé con `crontab -e` (línea `* /2 * * * *`) y lo confirmé con `crontab -l`.
4. Esperé las ejecuciones y leí el reporte con `cat ~/salud.txt`.

El código y el registro de la terminal están en los anexos.

3. Qué observé

Registro	RAM usada (MB)	RAM total (MB)	RAM (%)	Disco usado / total	Disco (%)	Carga 1 / 5 / 15 min
17:00:01	3463	15160	22,8	13G / 49G	28	0,94 / 0,87 / 0,78
17:02:01	3350	15160	22,1	13G / 49G	28	1,79 / 1,17 / 0,90

Tabla 1. Lo que quedó registrado en `salud.txt`.

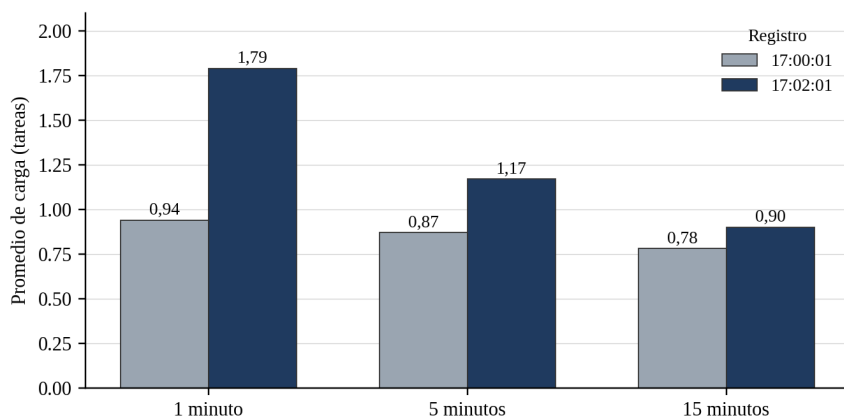


Figura 1. Promedios de carga en los dos registros.

Archivo	Permisos	Propietario	Grupo	Tamaño (bytes)	Modificado
salud.sh	-rwxrwxrwx	root	root	556	29 sep, 16:58
mundo.sh	-rwxrwxrwx	root	root	33	21 sep, 10:19

Tabla 2. Extracto de `ls -l`. La primera línea de la salida decía `total 44`.

Además: los dos registros están separados por exactamente 120 s y ambos empiezan un segundo después de iniciado el minuto; el disco no cambió; la RAM bajó 113 MB; y la carga subió en los tres promedios.

4. Por qué pasó

4.1 Cron y el segundo de retraso

cron es un *demonio*: un proceso que corre en segundo plano y cada minuto revisa si hay tareas que toquen. Si coinciden con la hora, hace un `fork()` y ejecuta el comando (Kerrisk, 2010; *cron(8)*). En `* / 2 * * * *`, el `* / 2` del campo de minutos significa 0, 2, 4, ..., 58; por eso salieron 17:00 y 17:02, justo lo esperado.

El segundo extra tiene sentido: Linux es un sistema de propósito general, no de tiempo real, así que el planificador no garantiza arrancar exactamente a tiempo (Tanenbaum y Bos, 2015), y además hay que crear el proceso, abrir el intérprete y llegar hasta `date`. Como fue igual las dos veces, parece un retraso sistemático, aunque con dos muestras no puedo asegurarlo. Para revisar cada dos minutos no importa. Sí vale recordar que *cron* corre con un entorno mínimo (por ejemplo, un `PATH` reducido, según *crontab(5)*); aquí funcionó porque `free`, `df`, `uptime` y `awk` están en esa ruta.

4.2 Un archivo de root con permisos 777

`salud.sh` quedó a nombre de *root* porque abrí `gedit` con `sudo`: un archivo nuevo pertenece al usuario efectivo del proceso que lo crea. En UNIX cada archivo tiene dueño, grupo y nueve bits de permiso (dueño, grupo, otros), y el núcleo aplica una sola clase: dueño si coincide, si no grupo, y si no, otros (Love, 2010). Yo caigo en «otros».

Con la máscara habitual (`umask 022`) el archivo habría nacido con 644, sin permiso de ejecución para mí. Con 777, «otros» tiene lectura, escritura y ejecución, y hacen falta las dos primeras: ejecución para `execve` y lectura para que el intérprete lea el código. Por eso el cron pudo correrlo. Los 556 bytes coinciden con el script del Anexo A.

Funciona, pero rompe el *principio de privilegio mínimo* (Saltzer y Schroeder, 1975): cualquier usuario podría modificar un archivo ejecutable. Lo correcto sería crearlo sin `sudo` y dejarlo en 755.

4.3 La RAM

La memoria usada bajó de 3463 a 3350 MB (22,8 % a 22,1 %): 113 MB, apenas el 0,75 % del total. Linux usa memoria virtual con paginación y una *caché de páginas* que se recupera cuando otros procesos la necesitan (Bovet y Cesati, 2005; Silberschatz et al., 2018), así que ese número se mueve todo el tiempo. Con estos datos no puedo saber la causa exacta; mis hipótesis son que algún proceso liberó memoria o que el núcleo recuperó caché. Para distinguirlas haría falta `ps`, `vmstat` o `/proc/meminfo`.

Dos detalles: el total de 15 160 MB es menor que los 16 384 MiB de un módulo de 16 GB, algo habitual porque el firmware, el núcleo y el vídeo integrado reservan memoria; y `free -m` reporta en mebibytes, así que la etiqueta «MB» de mi reporte es una simplificación.

4.4 El disco

El disco no se movió (13G de 49G, 28 %) porque cada registro pesa solo 134 bytes: 720 registros al día son unos 94 KiB, cerca de 34 MiB al año, y `df -h` redondea a gigabytes. El 28 % parece raro, ya que 13/49 es 26,5 %; pero `df` calcula usado / (usado + disponible) y el «disponible» excluye los bloques reservados para el administrador (en ext4, por defecto el 5 %, unos 2,45 GB). Sale consistente, aunque eso supone que el volumen es ext4 con la reserva por defecto, dato que no aparece en la evidencia.

El total 44 también se explica: el sistema de archivos asigna bloques de 4 KiB, así que nueve directorios (36) más dos archivos (8) dan 44. Incluso `undo.sh`, con 33 bytes, ocupa 4096 en disco: es la *fragmentación interna* (Silberschatz et al., 2018).

4.5 La carga del sistema

Los tres números de `uptime` son promedios exponencialmente amortiguados de las tareas activas (ejecutándose, listas o en espera ininterrumpible, como E/S de disco). El núcleo los actualiza cada 5 s (Walker, 2006; `proc(5)`):

$$L_t = L_{t-1} \cdot e^{-\Delta/\tau} + n \cdot (1 - e^{-\Delta/\tau}) \quad (1)$$

con n las tareas activas, $\Delta = 5$ s y $\tau = 60, 300$ o 900 s según el promedio. Esto explica tres cosas. La carga no es un porcentaje de CPU: 1,79 solo se interpreta comparándolo con el número de CPU lógicas (dato que no tengo; se ve con `nproc`). El promedio de 1 minuto reacciona más rápido, por eso subió más (+0,85) que el de 5 (+0,30) y el de 15 (+0,12). Y que en ambos registros se cumpla $1 \text{ min} > 5 \text{ min} > 15 \text{ min}$ indica que la actividad reciente supera a la histórica: carga creciente.

Para contrastar el modelo, si n hubiera sido constante durante los $T = 120$ s entre registros, despejo n en cada ventana:

$$n = (L_{\text{final}} - L_{\text{inicial}} \cdot e^{-T/\tau}) / (1 - e^{-T/\tau}) \quad (2)$$

Promedio	τ (s)	$e^{-T/\tau}$	L inicial	L final	n estimado	Margen por redondeo
1 min	60	0,1353	0,94	1,79	1,92	$\pm 0,01$
5 min	300	0,6703	0,87	1,17	1,78	$\pm 0,03$
15 min	900	0,8752	0,78	0,90	1,74	$\pm 0,08$

Tabla 3. Estimación de n con la ecuación (2). El margen viene de que los valores están redondeados a dos decimales.

Las tres estimaciones (1,92; 1,78 y 1,74) coinciden en que había cerca de dos tareas activas en esos dos minutos. Las diferencias caben en el margen por redondeo (mayor en la ventana de 15 minutos) y en que n no fue del todo constante. Que tres constantes de tiempo distintas apunten a la misma carga respalda el modelo. Eso sí, con dos registros no puedo decir que la carga sea sostenida: puede haber sido algo pasajero.

4.6 Las advertencias de gedit

Los mensajes de *Peas*, *Yaru* y *dconf* son avisos no fatales de componentes accesorios; el archivo se guardó igual. Los de *dconf* son los más instructivos: gedit guarda sus preferencias con GSettings, que usa el *bus de sesión D-Bus*; con *sudo* el proceso corre como *root*, con un entorno saneado y sin acceso al bus de mi sesión, y no encuentra *dbus-launch* para crear uno. Son diagnósticos probables, no verificados en el equipo. Tienen la misma raíz que la propiedad de *salud.sh*: abrir una aplicación gráfica completa como administrador. Mejor editar como usuario normal o usar *sudoedit*.

5. Qué mejoraría

- **Permisos y propiedad:** crear el archivo sin *sudo* y usar 755 en vez de 777.
- **Menos aperturas del archivo:** el script lo abre cinco veces por ejecución; agrupar con `{ ... ; }` `>> "$REPORTE"` lo reduce a una.
- **Que el reporte no crezca sin límite:** unos 34 MiB al año es poco, pero conviene rotarlo con *logrotate*.
- **Más datos:** muchas más muestras y guardar *nproc* junto con la carga, para juzgar tendencias y saturación.
- **Unidades y errores:** usar «MiB» en el reporte y mandar los errores a un archivo con `2>>`.
- **Alternativa a cron:** si el equipo se suspende, *cron* no recupera las ejecuciones perdidas; un temporizador de *systemd* con `Persistent=true` sí.

6. Conclusiones

- **Planificación:** las ejecuciones cayeron en minutos pares, como predice $*/2$; el segundo de retraso es normal en un sistema de propósito general.
- **Protección:** el archivo es de *root* por haber usado *sudo*, y solo pude ejecutarlo gracias a la clase «otros» con 777; funciona, pero no respeta el privilegio mínimo.

- **Memoria y disco:** los 113 MB de RAM son una variación normal (causa no determinable), y el disco no cambió por el tamaño mínimo del reporte, el redondeo de `df` y los bloques de 4 KiB.
- **Carga:** los tres promedios son consistentes con el amortiguamiento exponencial y apuntan a cerca de dos tareas activas.
- **Límite del experimento:** con dos muestras el análisis solo puede ser explicativo; más datos y las mejoras de la sección 5 lo harían más sólido.

Referencias

Bovet, D. P., y Cesati, M. (2005). *Understanding the Linux Kernel* (3.^a ed.). O'Reilly Media.

Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press.

Love, R. (2010). *Linux Kernel Development* (3.^a ed.). Addison-Wesley.

Saltzer, J. H., y Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308.

Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating System Concepts* (10.^a ed.). Wiley.

Tanenbaum, A. S., y Bos, H. (2015). *Modern Operating Systems* (4.^a ed.). Pearson.

Walker, R. (2006). Examining load average. *Linux Journal*.

Páginas del manual: `cron(8)`, `crontab(5)`, `free(1)`, `df(1)`, `uptime(1)`, `proc(5)`.

Anexo A. Código de salud.sh

Tal como se ejecutó (556 bytes):

```
#!/bin/bash
# El script se llama salud.sh y guarda el reporte en salud.txt
REPORTE="/home/cindy-guzman/salud.txt"

echo "=== $(date '+%Y-%m-%d %H:%M:%S') ===" >> "$REPORTE"

RAM_INFO=$(free -m | awk 'NR==2 {printf "RAM usada: %s MB de %s MB (%.1f%%)", $3, $2, $3*100/$2}')
echo "$RAM_INFO" >> "$REPORTE"

DISCO_INFO=$(df -h / | awk 'NR==2 {print "Disco usado: " $3 " de " $2 " (" $5 "%)"}')
echo "$DISCO_INFO" >> "$REPORTE"

CARGA_INFO=$(uptime | awk -F'load average:' '{print $2}')
echo "Carga del sistema:$CARGA_INFO" >> "$REPORTE"

echo "" >> "$REPORTE"
```

Anexo B. Registro de terminal (sin advertencias de gedit)

```
$ sudo gedit salud.sh
$ sudo chmod 777 salud.sh
$ ls -l
total 44
(directorios del usuario omitidos)
-rwxrwxrwx 1 root      root      33 Sep 21 10:19 mundo.sh
-rwxrwxrwx 1 root      root      556 Sep 29 16:58 salud.sh
$ crontab -e
crontab: installing new crontab
$ crontab -l
*/2 * * * * /home/cindy-guzman/salud.sh
$ cat ~/salud.txt
=== 2026-09-29 17:00:01 ===
RAM usada: 3463 MB de 15160 MB (22,8%)
Disco usado: 13G de 49G (28%)
Carga del sistema: 0,94, 0,87, 0,78

=== 2026-09-29 17:02:01 ===
RAM usada: 3350 MB de 15160 MB (22,1%)
Disco usado: 13G de 49G (28%)
Carga del sistema: 1,79, 1,17, 0,90
```